Lab2：基于 LLM 和 RAG 的多跳问题回答系统

赵楷越 522031910803

2025 年 4 月 29 日

1 普通 RAG 的系统搭建尝试

1.1 系统实现

在不适用 RAG 的情况下、直接将所有文本输入给大模型、最终得到的 EM 与 F1 指标如下图所示、作为本次实验的 baseline

```
================================================================================
Overall Scores:
================================================================================
Average EM: 0.1900 (19.00%)
Average F1: 0.2508 (25.08%)
================================================================================
Final Average EM: 0.1900, Final Average F1: 0.2508
Saving results to results\outputs.json...
Results saved successfully.
Evaluation complete!
```

1.1.1 尝试 1：分块 +BM25 检索

首先、我尝试了使用文档分块 +BM25 检索的方法，主要流程是：先加载数据集，将长文本切分成小段落；然后用 BM25 算法（基于 TF-IDF 和余弦相似度）从问题中提取关键词，检索出最相关的文本段落；接着将这些段落拼接后输入给 LLM 模型生成答案；最后用 EM 和 F1 分数评估答案准确性。

```python
# Chunking function
def chunk_context(context, chunk_size=500):
    words = context.split()
    chunks = [' '.join(words[i:i+chunk_size]) for i in range(0, len(words), chunk_size)]
    return chunks

# BM25 retrieval function
def bm25_retrieve(query, documents):
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(documents)
    query_vec = vectorizer.transform([query])
    similarities = cosine_similarity(query_vec, tfidf_matrix).flatten()
    most_relevant_indices = similarities.argsort()[-5:][::-1]
    return [documents[i] for i in most_relevant_indices]
```

结果如下图所示、发现不尽如人意。初步分析的主要缺点有：BM25 只是简单的词频统计，无法理解语义关系，对于需要多步推理的问题效果有限；检索时简单拼接问题和答案作为查询词可能引入偏差。因此、进行了后续的进一步代码实现。

```
----------------------------------------
================================================================================
Overall Scores:
================================================================================
Average EM: 0.1850 (18.50%)
Average F1: 0.3375 (33.75%)
================================================================================
Final Average EM: 0.1850, Final Average F1: 0.3375
Saving results to results\outputs.json...
Results saved successfully.
```

1.1.2 尝试 2：BERT Embedding+ 向量相似度查询

接着、我尝试使用了 BERT Embedding+ 向量相似度查询的方法。这个实现用 BERT 替换了之前的 BM25 检索，主要改进是：使用 BERT 模型将文本转换为语义向量，通过计算余弦相似度找出与问题最相关的段落；相比 BM25 的词频匹配，BERT 能捕捉更复杂的语义关系，对多跳问题中的隐含关联更有优势；实现上加载了 BERT 模型并放到 GPU 加速，对每个段落提取平均隐藏状态作为向量表示。初步分析的主要缺点包括：BERT 模型计算开销大，运行速度明显变慢落；BERT 的 512token 长度限制可能截断长文档。因此针对这个问题进行了进一步的改进。

```python
# Ensure CUDA is available and set device to GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Load tokenizer and model, move to GPU
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased').to(device)

def bert_embedding(text):
    inputs = tokenizer(text, padding=True, truncation=True, max_length=512, return_tensors='pt')
    inputs = inputs.to(device)
    outputs = model(**inputs)
    # Take the mean of the last hidden state along the sequence dimension
    embeddings = outputs.last_hidden_state.mean(dim=1)
    return embeddings.detach().cpu().numpy().squeeze(0)  # Squeeze the batch dimension

# ... (rest of the code remains the same)

def bert_retrieve(query, documents):
    query_embedding = bert_embedding(query)
    document_embeddings = [bert_embedding(doc) for doc in documents]
    similarities = cosine_similarity(query_embedding.reshape(1, -1), document_embeddings).flatten()
    most_relevant_indices = similarities.argsort()[-5:][::-1]
    return [documents[i] for i in most_relevant_indices]
```

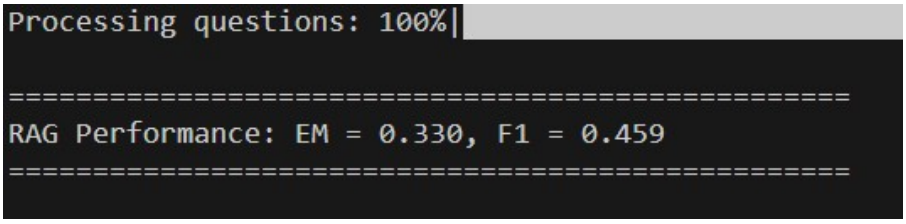
1.1.3 尝试 3：MiniLM Embedding + 向量相似度查询

最后、我尝试用轻量级的 all-MiniLM-L6-v2 模型替代了之前的 BERT，主要优化在于：首先在预处理阶段将长文本切分成大小为的 100 的 chunk 并设置 overlap=30 避免信息割裂、生成嵌入向量保存到本地、减少了计算量的同时使得每一次运行测试也不需要预先做 embedding；在检索时直接加载预处理的嵌入向量，通过余弦相似度快速找到与问题最相关的 15 个文本块。主要改进点包括：预处理嵌入避免重复计算；使用更高效的轻量模型；增加分块重叠保护上下文连贯性。分为 preprocess 和 main 代码、如下图所示

```python
# 更新为更现代的轻量级模型（无需均值池化警告）
model = SentenceTransformer('all-MiniLM-L6-v2')  # 替换原来的模型

def load_dataset(file_path="hotpotqa_longbench.json"):...

def chunk_text(text, chunk_size=100, overlap=30):...

def preprocess_and_save_embeddings():...

if __name__ == "__main__":
    preprocess_and_save_embeddings()
```

```python
# 初始化模型和客户端
model = SentenceTransformer('all-MiniLM-L6-v2')  # 与预处理使用相同的模型
client = OpenAI(base_url="http://47.242.151.133:12596/v1", api_key="abc123")

# --- 加载预处理数据 ---
def load_embeddings():...

# --- 检索相关文本块 ---
def retrieve_relevant_chunks(query, embeddings, chunk_info, top_k=3):...

# --- 模型交互 ---
def query_chat_model(messages, max_tokens=32):...

# --- 评估函数 ---
def normalize_answer(s):...
```

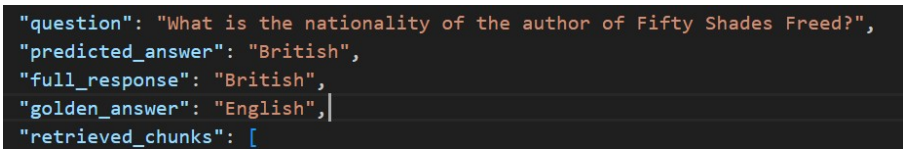
最终、该 RAG 系统的表现为 EM=0.330、F1=0.459，达到了预期的要求。



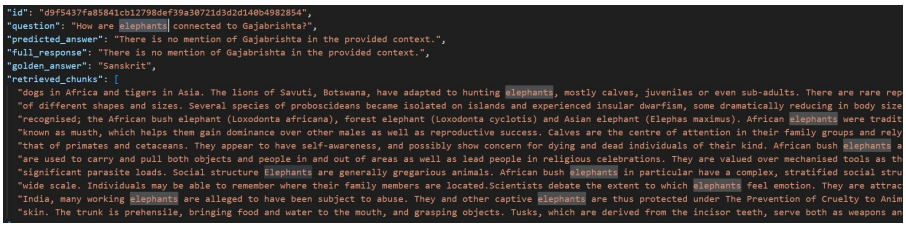
1.2 结果分析

同时、在进行测试的过程中、我同样对其中的 bad case 进行了案例分析、选取了几个典型的回答错误的例子、并尝试分析其原因。

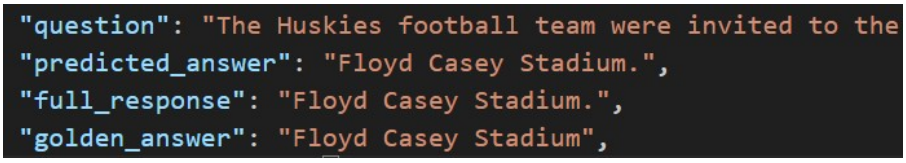
第一个例子、我们看到标准回答是 English、但是回答了同义词的 British、分析原因是因为在 retrieved chunks 中、既有 English 的表述又有 British 的表述。



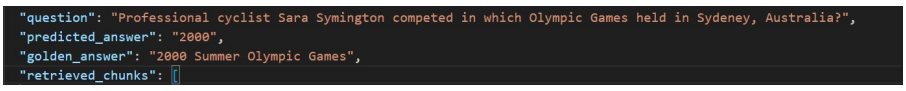
第二个例子、我们看到标准回答要求回答一个大象通过什么方式可以到达一座城市、但是 llm 的回答说它找不到信息、分析 retrieved chunks 后、发现其中检索出的相关信息都是相关问题中的关键词 elephants 的、根本不存在正确回答 Sanskrit 的内容。



第三个例子、标准回答没有句号、但是 llm 返回了最后的句号。不过这一点在对答案进行标准化的时候经过了处理。



第四个例子、即使是 llm 回答正确了答案、也会与标准答案不一致、这里 llm 选择回答了 2000 这个缩略的回答而不是 2000 年夏季奥运会。



2 设计的新策略

通过相关资料的搜索、以及对上面的具体 bad case 案例的分析、我进行了新策略的设计与尝试、主要分为以下几个步骤。

2.1 语义分块

首先、在 preprocess 阶段、采用了语义分块技术；相比之前简单的固定长度分块，语义分块能根据文本的实际含义自动划分段落，确保每个块在语义上是完整的；这样处理后的文本块更加便于 llm 去理解，避免了生硬切断句子或概念的情况；特别是在处理多跳问题时，语义连贯的文本块能更好地保留问题所需的逻辑关系；实际效果上，这种分块方式能让检索系统找到更相关的内容片段，减少无关信息的干扰。

```python
def semantic_chunk_text(text):
    # 使用兼容的嵌入模型
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    text_splitter = SemanticChunker(
        embeddings,
        breakpoint_threshold_type="percentile"
    )
    return text_splitter.split_text(text)
```

2.2 混合检索

接着、我采用了混合检索的方案、我把先前的 BM25 和向量检索结合起来一同使用；BM25 通过抓取包含关键词的精确匹配片段，来找那些直接包含问题中名词术语的段落；向量检索则通过语义理解找到意思相近但用词不同的相关内容；两个方法的分数先做归一化处理再按比例结合，这样既保留了关键词搜索的精准性，又具备了语义理解的灵活性；实际用起来比单独使用任一种方法效果都好，特别是处理需要同时考虑字面匹配和深层语义的多跳问题时优势明显；虽然计算上会慢一点，但能够有效提高检索质量。

```python
# --- Hybrid retrieval function ---
def retrieve_relevant_chunks(query: str,
                             embeddings: np.ndarray,
                             chunk_info: List[Dict],
                             bm25: BM25Okapi,
                             top_k: int = 3,
                             alpha: float = 0.5) -> List[Dict]:
    """
    Retrieve relevant chunks using hybrid BM25 + vector similarity approach

    Args:
        query: User query/question
        embeddings: Precomputed chunk embeddings
        chunk_info: Metadata about chunks
        bm25: Initialized BM25 model
        top_k: Number of chunks to return
        alpha: Weight for BM25 score (1-alpha for vector similarity)

    Returns:
        List of relevant chunks with combined scores, sorted by relevance
    """
```

2.3 提示词优化

然后、我也做了小幅度的提示词优化；通过要求严格基于上下文，并要求 llm 进行针对多跳问题的处理、让其进行角色扮演、最后输出与原来要求格式相同的结果、有效提升了性能。

```python
# 3. Construct multi-hop reasoning prompt
messages = [
    {
        "role": "system",
        "content": "You are an assistant specialized in answering multi-hop questions using provided context."
    },
    {
        "role": "user",
        "content": f"Context:\n{context}\n\nQuestion: {item['question']}\n\n"
                   "Instruction: Using ONLY the given context, provide a precise answer to this multi-hop question. "
                   "You should reason through the context and by delicately considering each part but don't output your reasoning content. "
                   "For Yes/No questions, respond strictly with 'Yes' or 'No'. "
                   "For all others, give only the exact required information (name, date, number, or brief phrase) "
                   "without forming complete sentences or adding explanations."
    }
]
```

2.4 其他策略分析与尝试

此外、我也分析并尝试了一些其他的策略；比如可以换成 deepseek 或 gpt4 这类性能更强的模型，保证更好的理解能力和推理质量；或者、针对多跳问题的特性，在检索前先对查询进行重写，比如把复杂问题拆解成几个子问题，这样检索时能更精准地找到需要的片段；再者、也可以在提示词设计上引入思维链的引导，让模型像人一样一步步推理；最后、还可以让模型自己判断是否需要继续多跳思考，复杂的再分步处理。

这个实验中、我初步尝试实现了一下查询重写的流程、如下图所示、也附在了 code_version 文件夹中、不过最后的性能表现并不符合预期。

```python
# --- Query Decomposition ---
def decompose_query(query: str) -> Tuple[List[str], str]:
    """
    Decompose complex query into sub-questions using LLM

    Args:
        query: Original complex question

    Returns:
        Tuple of (sub_questions, reasoning_steps)
    """
    decomposition_prompt = [
        {
            "role": "system",
            "content": "You are an expert at breaking down complex questions into simpler sub-questions. "
                       "Identify the key components needed to answer the main question."
        },
```

2.5 结果分析及文献阅读

最终在新策略的版本上、实现的 Hybrid RAG 系统的性能相较于先前有了小幅度的提升。

```
==================================================
Hybrid RAG Performance: EM = 0.350, F1 = 0.463
==================================================
```

通过研究相关文献，我也对提升大语言模型（LLM）在多跳问题上的回答质量有了更深入的认识。多跳问答的核心挑战在于如何高效检索分散在多个文档中的证据片段，并实现连贯的推理。最近的研究提出了多种创新方案：比如 Zhang 团队提出的 Beam Retrieval 框架（2023），通过端到端建模多跳检索过程，并引入束搜索机制保留多个候选路径，显著降低了漏检风险，在 MuSiQue-Ans 数据集上性能提升近 50%；而 BigBird 模型（Zaheer 等，2020）则通过稀疏注意力机制突破序列长度限制，使模型能处理更长上下文，这对需要跨文档关联的多跳推理尤为重要。在动态检索策略方面，Zhu 等（2021）的 AISO 方法将检索过程建模为马尔可夫决策过程，能根据问题复杂度自适应选择 BM25、DPR 等不同检索器。

本次实验的给我带来的收获颇多，让我更具体地认识了 llm 中的多跳问题这一研究方向。